

ALGORITMI E STRUTTURE DATI

ALGORITMI SU GRAFI

AULA VIRTUALE DEL 18-05-2026

V 1.0

ALGORITMI SU GRAFI

Un grafo è una **nozione matematica astratta**, usata per rappresentare l'idea di una *«connessione»* o di *«relazione»* tra coppie di oggetti. In particolare:

Un grafo G = (V, E) consiste in:

- un insieme V di «vertici» o («nodi»), e
- un insieme E di coppie di vertici, detti «archi» (o «spigoli»); ogni arco connette due vertici.

Notiamo che in questa definizione astratta non è importante cosa sia rappresentato dai vertici, dagli archi o complessivamente dal grafo; questo aspetto dipende in generale dal problema o dall'applicazione che si vuole modellare.

Esistono due tipologie di grafi: grafi **orientati** e grafi **non orientati**. In un gruppo non orientato (oppure **diretto**) gli archi non hanno un'orientazione: ciò accade quando la relazione tra una coppia di vertici esemplificata da un arco è simmetrica.

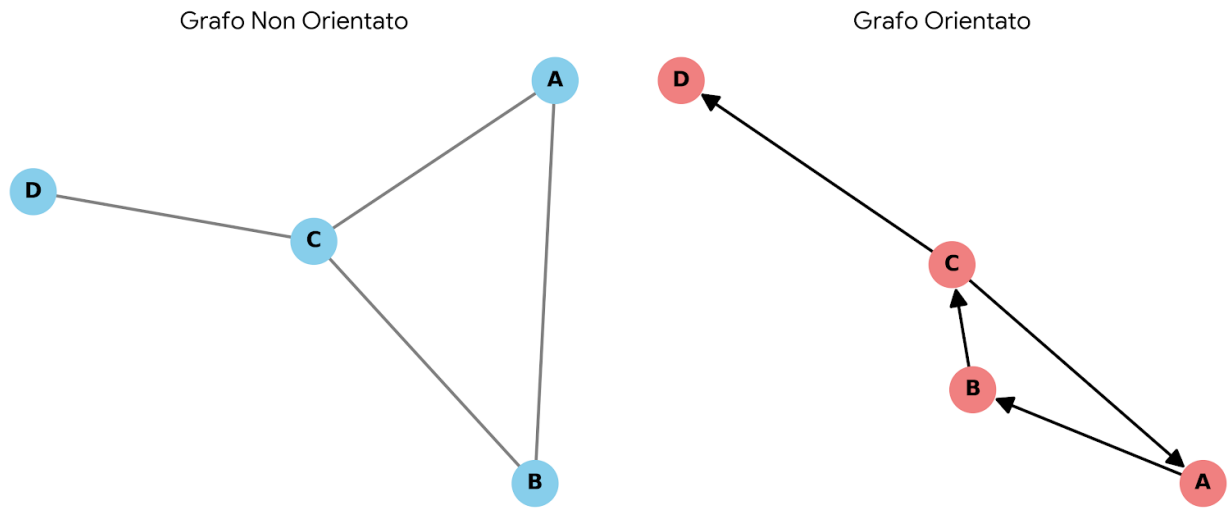
ALGORITMI SU GRAFI

```
import networkx as nx
import matplotlib.pyplot as plt

# Creazione dei sotto-grafi affiancati
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

# 1. Configurazione del Grafo Non Orientato
# In un grafo non orientato, le relazioni sono simmetriche: {A, B} equivale a {B, A}
G_undirected = nx.Graph()
G_undirected.add_edges_from([('A', 'B'), ('A', 'C'), ('B', 'C'), ('C', 'D')])

pos1 = nx.spring_layout(G_undirected, seed=42) # Layout per la disposizione dei nodi
nx.draw(G_undirected, pos1, ax=ax[0], with_labels=True,
        node_color='skyblue', node_size=900, edge_color='gray',
        font_size=14, font_weight='bold', width=2)
ax[0].set_title("Grafo Non Orientato", fontsize=16, fontweight='bold')
```

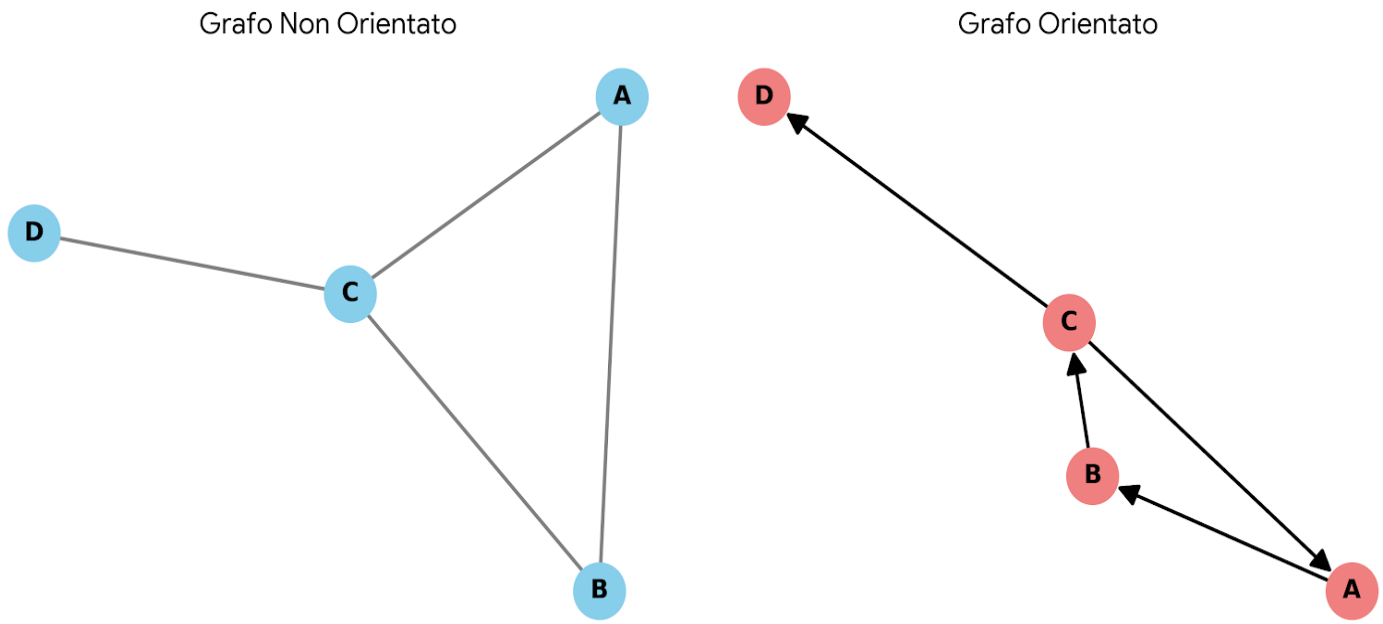


ALGORITMI SU GRAFI

```
# 2. Configurazione del Grafo Orientato
# In un grafo orientato, l'ordine delle coppie conta: (A, B) rappresenta la freccia da A a B
G_directed = nx.DiGraph()
G_directed.add_edges_from([('A', 'B'), ('B', 'C'), ('C', 'A'), ('C', 'D')])

pos2 = nx.spring_layout(G_directed, seed=42)
nx.draw(G_directed, pos2, ax=ax[1], with_labels=True,
        node_color='lightcoral', node_size=900, edge_color='black',
        font_size=14, font_weight='bold', arrows=True, arrowsize=25, width=2)
ax[1].set_title("Grafo Orientato", fontsize=16, fontweight='bold')

# Ottimizzazione del layout e salvataggio dell'immagine
plt.tight_layout()
plt.savefig('grafi_confronto.png', dpi=300)
```



ALGORITMI SU GRAFI

Molte delle definizioni di base sui grafi sono comuni a grafi orientati e non orientati. A volte però, nel caso di grafi orientati, si hanno piccole differenze di notazione. Nelle seguenti definizioni, faremo riferimento sia a grafi orientati che a grafi non orientati, a meno che non sia specificato esplicitamente.

Adiacenza e incidenza. Se (x,y) è l'arco di un grafo G, diciamo che (x,y) è **incidente** sui vertici x e y. Se inoltre il grafo G è orientato, diciamo che l'arco (x,y) **esce** dal vertice x ed **entra** nel vertice y. Nel caso di un grafo non orientato, diremo che x è **adiacente** a y, y è **adiacente** a x, ovvero x ed y sono **adiacenti**. Nel caso di grafo orientato, diremo che y è **adiacente** a x. Dato un vertice v, chiameremo i vertici adiacenti a v anche vicini a v.

Grado di un vertice. Il **grado** di un vertice v in un grafo è dato dal numero di archi incidenti su v. Denoteremo il grado del vertice v con la notazione δ(v). Osserviamo che, se sommiamo tutti i gradi dei vertici di un grafo non orientato, conteremo ogni arco (x,y) esattamente due volte: una volta quando consideriamo il grado di x ed una volta quando consideriamo il grado di y. Quindi: $\sum_{v \in G} \delta$ (v) = 2m.

ALGORITMI SU GRAFI

Se il grafo è orientato, parleremo di grado di uscita di v, intendendo con questo il numero di archi che escono da v, e di grado in entrata di v, intendendo con questo il numero di archi che entrano in v. Denoteremo il grado di entrata ed il grado di uscita di un vertice v, rispettivamente con la notazione $\delta_{in}(v)$ e $\delta_{out}(v)$. Nel caso di un grafo orientato, il grado di un vertice sarà dato dal suo grado in entrata sommato al grado di uscita: δ(v) = $\delta_{in}(v)$ + $\delta_{out}(v)$.

Cammini e cicli. Un cammino in un grafo G da un vertice x ad un vertice y è dato da una sequenza di vertici (v_0 , v_1 , **…,** v_k) con v_0 =x e v_k = y, tale che, per 1 <= i <= k, l'arco (v_{i-1} , v_i) appartiene a G. In particolare, in tal caso diremo che il cammino è di **lunghezza** k. Diremo che il cammino **contiene** (ovvero **passa per**) i vertici v_0 , v_1 ,…, v_k e gli archi (v_0 , v_1), (v_1 , v_2), … , (v_{k-1} , v_k). Diremo che il cammino è **semplice** se tutti i suoi vertici sono distinti. Nel caso di grafi orientati, indicheremo un cammino dal vertice x al vertice y con la notazione x ⇝ y.

ALGORITMI SU GRAFI

Se esiste un cammino da un vertice x ad un vertice y di G, diremo anche che y è raggiungibile da x, ovvero che y è un discendente di x e che x è un antenato di y. Un cammino $\langle v_0 , v_1 ,\ldots, v_k \rangle$ tale che $v_0 = v_k$ e k>=1 è un ciclo. Il ciclo è semplice se i vertici $v_0 , v_1 ,\ldots, v_{k-1}$ sono tutti distinti. **Un grafo diretto aciclico è un grafo orientato non contenente cicli**.

Connettività e connettività forte. Un grafo non orientato G=(V, E) si dice connesso se esiste un cammino tra ogni coppia di vertici in G.
Un grafo orientato G = (V, E) si dice **fortemente connesso** se esiste un cammino (orientato) tra ogni coppia di vertici in G.
Diremo inoltre che un vertice x è fortemente connesso ad un vertice y se esiste un cammino (orientato) da x ad y ed un cammino (orientato) da y ad x. Notiamo che i due cammini possono anche avere archi o vertici in comune, o possono essere anche cammini degeneri, come quelli contenenti un solo vertice: come caso particolare, un vertice è fortemente connesso a se stesso. Utilizzeremo la notazione **x** ⇔ **y** per indicare che il vertice x è fortemente connesso a y.

ALGORITMI SU GRAFI

Sottografi. Diciamo che un grafo $G' = (V', E')$ è un sottografo di un grafo G = (V, E) se $V' \subseteq$ V ed $E' \subseteq$ E. In questo caso utilizzeremo la notazione $G' \subseteq$ G. Dati un grafo G = (V, E) ed un insieme di vertici $V' \subseteq$ V, il sottografo di G indotto da V' è definito come il grafo $G' = (V', E')$ dove $E' = \{(x, y) \in E : x,y \in V'\}$.

E' da notare da queste definizioni preliminari, i grafi hanno una struttura molto ricca e complessa. Non sorprende quindi che sia possibile definire un tipo di dato grafo in vari modi, soprattutto in funzione delle operazioni che intendiamo supportare su di esso.

ALGORITMI SU GRAFI

Per essere efficacemente interpretata da un programma, una rappresentazione di un grafo deve essere vicina alla sua definizione astratta. Vediamo ora i principali metodi di rappresentazione di un grafo.

Rappresentazione di grafi con lista di archi. Possiamo definire una qualche struttura per ogni vertice, che rappresenta l'informazione che vogliamo associare ad un vertice, ed un'altra struttura per ogni arco, contenente puntatori ai due vertici estremi dell'arco. In pratica una tale rappresentazione avrà una lista o un array di strutture, dove utilizzeremo una struttura per ogni vertice ed una per ogni arco. Faremo, quindi, riferimento a questo modo di rappresentare un grafo come rappresentazione con lista di archi. Lo spazio totale usato da questa rappresentazione è O(m+n), visto che serve una quantità costante di spazio (una struttura) per ogni vertice e per ogni arco.
Anche se questa rappresentazione è molto diffusa, ci sono alcune operazioni che non sembrano facilmente realizzabili in modo efficiente. Si pensi ai tempi di esecuzione della rappresentazione di un grafo non orientato rappresentato con lista di archi, il punto di debolezza sembra infatti che molte operazioni e delle tipologie di accessi agli archi del grafo richiedono l'esame dell'intera lista di archi, quindi tempo O(m) nel caso peggiore.

ALGORITMI SU GRAFI

Rappresentazione di grafi con liste di adiagenza. Una rappresentazione più utilizzata della precedente è quella con liste di adiacenza. In tale rappresentazione, ogni vertice v ha una lista contenente i suoi vertici adiacenti, ovvero tutti i vertici u per cui esiste un arco (v,u). Notiamo che gli archi non appaiono esplicitamente in questa rappresentazione, ma sono codificati nella nozione di adiacenza.

Con questa rappresentazione diventa più semplice trovare gli archi incidenti su un particolare vertice v, per far questo è sufficiente scandire la lista di adiacenza di v. Osserviamo anche che ci sono n liste di adiacenza, una per ogni vertice. Nel caso di grafi non orientati la lunghezza totale delle liste di adiacenza è esattamente 2m, visto che ogni arco (x,y) è rappresentato due volte, una nella lista di adiacenza di x e l'altra nella lista di adiacenza di y. Nel caso di grafi orientati la lunghezza totale delle liste di adiacenza è invece m, visto che ogni arco (x,y) è rappresentato una sola volta, ovvero nella lista di adiacenza di x. In entrambi i casi, lo spazio richiesto dalla rappresentazione con liste di adiacenza è ancora O(m+n).

ALGORITMI SU GRAFI

Rappresentazione di grafi con liste di incidenza. La rappresentazione con liste di incidenza combina le caratteristiche ed i vantaggi della rappresentazione con lista di archi descritta prima e della rappresentazione con liste di adiacenza. In particolare, in aggiunta alla rappresentazione con lista di archi, basata su strutture per ogni arco e per ogni vertice, memorizziamo per ogni vertice v una lista di puntatori agli archi incidenti a v. La quantità di spazio richiesta è leggermente superiore a quella delle precedenti rappresentazioni, ma è ancora O (m+n) nel caso peggiore.

Rappresentazione di grafi con matrici di adiacenza. In alcune applicazioni potremmo voler privilegiare una rappresentazione del grafo che richieda più spazio, ma per cui la verifica della presenza di un arco possa essere effettuata più velocemente. In questi casi, possiamo utilizzare una rappresentazione del grafo tramite matrice di adiacenza. Per comodità, assumiamo che i vertici del grafo che dobbiamo rappresentare corrispondano ai numeri interi da 1 a n. Se non è così osserviamo che è sempre possibile definire una corrispondenza biunivoca tra gli n vertici di un grafo e gli interi da 1 a n. Sia quindi M una matrice, di dimensione n x n, le cui righe e colonne sono «indicizzate» dai vertici del grafo. La matrice di adiacenza M è definita così: M[u,v] = { 1 se (u,v) è un arco di G; 0 altrimenti }

ALGORITMI SU GRAFI

Rappresentazione di grafi con matrici di incidenza. La matrice di incidenza è una matrice di dimensione n x m: le righe della matrice sono «indicizzate» dai vertici mentre le colonne sono «indicizzate» dagli archi. Similmente al caso della matrice di adiacenza, la matrice di incidenza avrà un valore uguale ad 1 quando arco e vertice corrispondenti sono incidenti. Di conseguenza, ogni colonna ha esattamente due 1: in particolare, la colonna corrispondente all'arco (x,y) avrà valore uguale ad 1 solamente nelle due righe corrispondenti ad x e ad y. Nel caso di un grafo orientato, possiamo usare una matrice simile, in cui ogni colonna ha esattamente un valore +1 ed un valore -1 per distinguere tra il vertice sorgente ed il vertice di destinazione dell'arco. Da osservare che la matrice di incidenza non è necessariamente una matrice quadrata, e quindi neanche simmetrica.

ALGORITMI SU GRAFI

La visita di un grafo (o attraversamento) è l'operazione che permette di esplorare sistematicamente tutti i nodi (e gli archi) di un grafo partendo da un nodo iniziale specifico.
Le due strategie fondamentali e più utilizzate per visitare un grafo sono la **Visita in Ampiezza (BFS)** e la **Visita in Profondità (DFS)**. Entrambe si applicano sia a grafi orientati che non orientati.

ALGORITMI SU GRAFI

Visita in Ampiezza (BFS - Breadth-First Search)

La visita in ampiezza esplora il grafo "a livelli". Partendo dal nodo iniziale, si visitano prima tutti i suoi vicini diretti (distanti 1 arco), poi i vicini dei vicini (distanti 2 archi), e così via. Si sviluppa in modo orizzontale.

- Struttura dati utilizzata: Una Coda (FIFO - First In, First Out).
- Logica: Esplora i nodi più vicini alla sorgente prima di passare a quelli più lontani.
- Caso d'uso ideale: Trovare il cammino minimo (minor numero di archi) tra il nodo di partenza e un qualsiasi altro nodo in un grafo non pesato.

Esempio pratico:
Se parti dal nodo A, la BFS visiterà prima tutti i figli di A, poi i nipoti di A, espandendosi come le onde generate da un sasso lanciato nello stagno.

ALGORITMI SU GRAFI

Visita in Profondità (DFS - Depth-First Search)

La visita in profondità esplora il grafo "per rami". Partendo dal nodo iniziale, si seleziona un percorso e lo si segue fino in fondo (fino a quando non si incontra un nodo senza vicini non visitati o un vicolo cieco). A quel punto si torna indietro (backtracking) e si esplorano le diramazioni lasciate in sospeso. Si sviluppa in modo verticale.

- Struttura dati utilizzata: Uno Stack (LIFO - Last In, First Out), spesso implementato implicitamente tramite la ricorsione.
- Logica: Prosegue in avanti lungo un cammino il più possibile prima di effettuare il backtracking.
- Caso d'uso ideale: Rilevamento di cicli nel grafo, ordinamento topologico, o verifica della connettività (es. trovare le componenti fortemente connesse).

Esempio pratico:
Se ti trovi in un labirinto, la DFS equivale a scegliere una strada e seguirla fino a un muro, per poi tornare indietro fino all'ultimo bivio e tentare l'altra direzione.

ALGORITMI SU GRAFI

Tabella Comparativa tra BFS e DFS

Caratteristica	BFS (In Ampiezza)	DFS (In Profondità)
Struttura Dati	Coda (Queue)	Stack / Ricorsione
Strategia	Esplora per livelli successivi	Esplora un ramo fino in fondo
Cammino Minimo	Sì (in grafi non pesati)	No, non garantisce il cammino più breve
Spazio di Memoria	Richiede più memoria se il grafo è molto largo	Richiede più memoria (nello stack) se il grafo è molto profondo
Applicazione tipica	Reti peer-to-peer, GPS, social network (amici comuni)	Soluzione di puzzle/labirinti, analisi di dipendenze (compilatori)

ALGORITMI SU GRAFI

Complessità Computazionale

Per entrambe le tipologie di visita, se il grafo è rappresentato tramite una lista di adiacenza, la complessità temporale è la medesima:
$O(|V| + |E|)$

Dove:
$|V|$ è il numero di vertici (nodi).
$|E|$ è il numero di archi (collegamenti).

Questo accade perché ogni nodo e ogni arco viene esaminato un numero costante di volte durante l'intero processo di attraversamento.

ALGORITMI SU GRAFI

Algoritmo di Dijkstra

Dijkstra adotta una strategia di tipo greedy. Funziona selezionando passo dopo passo il nodo non ancora visitato che si trova alla distanza minima provvisoria dalla sorgente, "rilassando" poi tutti i suoi archi uscenti.

Punto di forza: È estremamente veloce ed efficiente.
Grande limite: Non funziona se il grafo contiene archi con pesi negativi. Se è presente anche un solo arco negativo,

Dijkstra può calcolare percorsi errati perché presuppone che aggiungere un arco aumenti (o lasci invariato) il costo del cammino, il che non è vero con pesi negativi.

Struttura dati: Utilizza una coda con priorità (Min-Heap) per selezionare rapidamente il nodo più vicino.

ALGORITMI SU GRAFI

```python
import heapq

def dijkstra(grafo, partenza):
    """
    Calcola i cammini minimi da un vertice di partenza a tutti gli altri nodi
    in un grafo con pesi non negativi.

    :param grafo: Dizionario che rappresenta la lista di adiacenza del grafo.
                  Es: {'A': {'B': 2, 'C': 4}, ...}
    :param partenza: Il nodo da cui iniziare la ricerca.
    :return: Un dizionario con le distanze minime da 'partenza' a ogni nodo.
    """

    # Inizializziamo tutte le distanze come infinite (inf)
    # tranne il nodo di partenza che ha distanza 0 da se stesso.
    distanze = {nodo: float('inf') for nodo in grafo}
    distanze[partenza] = 0

    # Creiamo la coda di priorità (min-heap) e inseriamo il nodo iniziale.
    # La tupla contiene (distanza_corrente, nodo)
    coda_priorita = [(0, partenza)]
```

ALGORITMI SU GRAFI

```
while coda_priorita:
    # Estraiamo il nodo con la distanza minore memorizzata finora
    distanza_attuale, nodo_attuale = heapq.heappop(coda_priorita)

    # Se abbiamo già trovato un cammino più breve per questo nodo,
    # saltiamo l'elaborazione (evita calcoli ridondanti)
    if distanza_attuale > distanze[nodo_attuale]:
        continue

    # Esaminiamo i vicini del nodo attuale (concetto di adiacenza)
    for vicino, peso in grafo[nodo_attuale].items():
        # Calcoliamo la distanza potenziale passando per il nodo attuale
        distanza_alternativa = distanza_attuale + peso

        # Se il nuovo cammino è più breve di quello precedentemente registrato
        if distanza_alternativa < distanze[vicino]:
            # Aggiorniamo la distanza minima per il vicino
            distanze[vicino] = distanza_alternativa
            # Inseriamo il vicino nella coda di priorità con la nuova distanza
            heapq.heappush(coda_priorita, (distanza_alternativa, vicino))

return distanze
```

ALGORITMI SU GRAFI

```
# --- Esempio di utilizzo pratico ---

# Definiamo un grafo non orientato usando le liste di adiacenza pesate.
# Ogni chiave è un vertice e il valore è un altro dizionario con {vicino: peso_arco}.
grafo_esempio = {
    'A': {'B': 4, 'C': 2},
    'B': {'A': 4, 'C': 1, 'D': 5},
    'C': {'A': 2, 'B': 1, 'D': 8, 'E': 10},
    'D': {'B': 5, 'C': 8, 'E': 2, 'Z': 6},
    'E': {'C': 10, 'D': 2, 'Z': 3},
    'Z': {'D': 6, 'E': 3}
}

# Nodo di partenza scelto per il test
nodo_iniziale = 'A'
risultato_distanze = dijkstra(grafo_esempio, nodo_iniziale)

# Stampa dei risultati a schermo
print(f"Distanze minime partendo dal nodo '{nodo_iniziale}':")
for nodo, distanza in risultato_distanze.items():
    print(f"Distanza fino al nodo {nodo}: {distanza}")
```

ALGORITMI SU GRAFI

L’algoritmo di Bellman-Ford

Si basa sulla programmazione dinamica. Invece di scegliere il nodo più vicino in modo "ingordo", l'algoritmo adotta un approccio a forza bruta controllata: effettua un ciclo di "rilassamento" di tutti gli archi del grafo, e ripete questo intero processo per |V| - 1 volte (dove |V| è il numero di nodi).

Punto di forza: Gestisce senza problemi archi con pesi negativi.

Rilevamento di cicli negativi: È in grado di scoprire se nel grafo esiste un "ciclo a costo negativo" (un loop che continua a ridurre il costo all'infinito). Se esegue un ulteriore controllo (un V-esimo ciclo) e le distanze continuano a ridursi, l'algoritmo segnala che il problema non ha soluzione perché il costo potrebbe scendere all'infinito.

Grande limite: È molto più lento di Dijkstra.

ALGORITMI SU GRAFI

```python
def bellman_ford(n, edges, source):
    """
    Calcola i cammini minimi da una sorgente usando Bellman-Ford.

    Parametri:
    n      = numero di vertici, numerati da 0 a n - 1
    edges  = lista degli archi nel formato (u, v, peso)
    source = vertice sorgente

    Ritorna:
    dist = lista delle distanze minime dalla sorgente
    pred = lista dei predecessori, utile per ricostruire i cammini

    Se esiste un ciclo negativo raggiungibile dalla sorgente,
    ritorna None.
    """

    # Inizializzo tutte le distanze a infinito
    dist = [float("inf")] * n

    # Il predecessore di ogni nodo è inizialmente sconosciuto
    pred = [None] * n

    # La distanza della sorgente da sé stessa è 0
    dist[source] = 0
```

ALGORITMI SU GRAFI

```
# Ripeto il rilassamento degli archi n - 1 volte
for _ in range(n - 1):
    modificato = False

    for u, v, peso in edges:
        # Se u è raggiungibile e passando da u miglioro la distanza di v
        if dist[u] != float("inf") and dist[u] + peso < dist[v]:
            dist[v] = dist[u] + peso
            pred[v] = u
            modificato = True

    # Ottimizzazione: se in un'intera iterazione non cambia nulla,
    # le distanze sono già definitive
    if not modificato:
        break

# Controllo la presenza di cicli negativi
# Se riesco ancora a migliorare una distanza,
# allora esiste un ciclo negativo raggiungibile dalla sorgente
for u, v, peso in edges:
    if dist[u] != float("inf") and dist[u] + peso < dist[v]:
        return None, None

return dist, pred
```

ALGORITMI SU GRAFI

```
n = 5

edges = [
    (0, 1, 6),
    (0, 2, 7),
    (1, 2, 8),
    (1, 3, 5),
    (1, 4, -4),
    (2, 3, -3),
    (2, 4, 9),
    (3, 1, -2),
    (4, 3, 7),
    (4, 0, 2)
]

source = 0

dist, pred = bellman_ford(n, edges, source)

if dist is None:
    print("Il grafo contiene un ciclo negativo raggiungibile dalla sorgente.")
else:
    print("Distanze minime dalla sorgente:")
    for v in range(n):
        print(f"Vertice {v}: distanza = {dist[v]}")
```

ALGORITMI SU GRAFI

Confronto Diretto

Caratteristica	Algoritmo di Dijkstra	Algoritmo di Bellman-Ford
Approccio	Greedy (Ingordo)	Programmazione Dinamica
Pesi Negativi	Non supportati (restituisce risultati errati)	Supportati correttamente
Cicli Negativi	Non li rileva (va in loop o fallisce)	**Li rileva** e segnala l'errore
Complessità Temporale	$O((\|V\| + \|E\|) \log$ *(con Min-Heap)*	$O(\|V\| \cdot \|$
Caso d'uso tipico	Routing di rete (OSPF), Mappe/GPS (Google Maps)	Protocolli di routing Distance-Vector (RIP), grafi finanziari (arbitraggio valutario)

ALGORITMI SU GRAFI

Alberi Ricoprenti Minimi (Minimum Spanning Tree - MST)

Immagina di dover collegare un gruppo di case con una rete in fibra ottica: vuoi che tutte le case siano connesse tra loro, ma vuoi usare la minor quantità di cavo possibile per ridurre i costi. Il risultato ottimale è un Albero Ricoprente Minimo.

- Albero: un sottografo connesso e privo di cicli (non ci sono percorsi chiusi inutili).
- Ricoprente: include tutti i nodi del grafo originale.
- Minimo: la somma dei pesi degli archi scelti è la più bassa possibile.

Per risolverlo, possiamo ricorrere a due storici algoritmi di tipo **greedy**.

ALGORITMI SU GRAFI

Sia dato un grafo non orientato e connesso G = (V, E), con m archi ed n vertici deve essere valida la nozione di m >= n – 1. Introduciamo ora la nozione di albero ricoprente.

Definizione Albero ricoprente: dato un grafo G = (V, E) non orientato e connesso, un albero ricoprente di G è un sottografo T ⊆ G tale che:

(i) T è un albero, e
(ii) T contiene tutti i vertici di G.

Supponiamo ora che ogni arco *e* di *G* abbiano associato un costo (o peso) w(*e*). In modo più formale, assumiamo che sia definita una funzione costo w: E ⟶ R. In tal caso, possiamo definire il costo di un albero ricoprente T di G come la somma dei costi dei suoi archi:

$$w(T) = \sum_{e \in T} w(e)$$

Alberti ricoprenti diversi avranno in genere costi diversi.

ALGORITMI SU GRAFI

Definizione di Minimo albero ricoprente: Dato un grafo G = (V , E) non orientato, connesso e pesato sugli archi, un minimo albero ricoprente di G è un albero ricoprente di G di costo minimo.

Trovare un minimo albero ricoprente è probabilmente uno tra i problemi di ottimizzazione più studiati in letteratura: la prima pubblicazione scientifica sul minimo albero ricoprente risale infatti al 1926 [1]. In generale la soluzione non è unica: uno stesso grafo potrebbe ammettere più di un minimo albero ricoprente.

Per risolvere il problema del minimo albero ricoprente utilizzeremo la tecnica algoritmica **greedy**.

Definizione: un algoritmo **greedy** risolve un problema effettuando una sequenza di scelte locali ottime, con l'obiettivo che tali scelte conducano a una **soluzione globale ottima**.

[1] Boruvka, O. (1926). O Jistem problemu minimalim. Maravské Prirodoverecke Spolelecnosti. 3, 37-58

ALGORITMI SU GRAFI

L’idea chiave alla base della definizione di algoritmi greedy, è quindi:

In pratica, scegli **ora** la **soluzione migliore disponibile**, sperando che questa porti alla **soluzione migliore complessiva**.

Un algoritmo **greedy** è corretto solo per problemi che soddisfano proprietà particolari, soprattutto:

1. Proprietà della scelta greedy: esiste una soluzione ottima che contiene la scelta localmente ottima fatta al primo passo.
2. Sottostruttura ottima: dopo aver fatto una scelta greedy, il problema rimanente è ancora dello stesso tipo e la sua soluzione ottima può essere combinata con la scelta fatta.

ALGORITMI SU GRAFI

Proprietà dei minimi alberi ricoprenti

Per risolvere il problema del MST, utilizzeremo alcune tecniche algoritmiche greedy. Nello specifico, descriveremo la tecnica greedy come un processo di colorazione [2]. All'inizio tutti gli archi **non** saranno colorati. Durante il processo, coloreremo un arco alla volta: quando coloreremo un arco blu, lo includeremo nella soluzione: quando coloreremo un arco rosso, lo escluderemo dalla soluzione. Per specificare meglio il processo di colorazione, abbiamo bisogno della notazione di taglio in un grafo.

Definizione (Taglio di un grafo): Dato un grafo non orientato G = (V, E), un taglio in G è una partizione dei vertici V in due insiemi: X e $\bar{X}$ = V – X. Un arco e = (u,v) attraversa il taglio (X, $\bar{X}$) se u ∈ X e v ∈ $\bar{X}$.

ALGORITMI SU GRAFI

Proprietà dei minimi alberi ricoprenti

Presenteremo due regole che ci saranno utili nel delineare il paradigma di tecnica greedy per il MST.

Regola del taglio: Scegli un taglio di G che non contiene archi blu. Tra tutti gli archi non colorati del taglio, seleziona un arco di costo minimo e coloralo di blu.

Regola del ciclo: Scegli un ciclo semplice in G che non contiene archi rossi. Tra tutti gli archi non colorati del ciclo, seleziona un arco di costo massimo e coloralo rosso.

Quindi il metodo greedy applica una qualsiasi delle due regole ad ogni passo, e termina quando riesce a colorare tutti gli archi.
Teorema: Sia G un grafo orientato, connesso, e pesato sugli archi. Ogni passo di colorazione del metodo greedy mantiene la seguente invariante: «Esiste sempre un minimo albero ricoprente di G che contiene tutti gli archi blu, e che non contiene alcun arco rosso.». Inoltre, il metodo greedy colora tutti gli archi di G.

ALGORITMI SU GRAFI

Algoritmo di Kruskal

L’algoritmo in questione ordina tutti gli archi del grafo dal più leggero al più pesante. Prende l'arco più piccolo e lo inserisce nell'albero, a patto che non crei un ciclo con gli archi già scelti. Ripete l'operazione fino a collegare tutti i nodi.

Struttura dati correlata (Union-Find): Per verificare in modo fulmineo se l'aggiunta di un arco crea un ciclo, Kruskal si appoggia alla **struttura dati Union-Find** (detta anche Disjoint-Set). Questa struttura raggruppa i nodi in sottoinsiemi disgiunti e permette di capire istantaneamente se due nodi appartengono già alla stessa "componente" dell'albero.

Sulla base del teorema appena enunciato, gli archi blu formano una foresta di n alberi blu disgunti. Durante la sua esecuzione, l’algoritmo di Kruskal mantiene la foresta di alberi blu. Se l’arco ha entrambi gli estremi nello stesso albero blu, coloralo di rosso. Altrimenti, coloralo di blu.

ALGORITMI SU GRAFI

```
class UnionFind:
    def __init__(self, n):
        # Ogni nodo è inizialmente il rappresentante di sé stesso
        self.parent = list(range(n))

        # rank serve per mantenere gli alberi il più possibile bassi
        self.rank = [0] * n

    def find(self, x):
        # Trova il rappresentante dell'insieme a cui appartiene x
        # Compressione del cammino: rende più veloci le ricerche future
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])

        return self.parent[x]
```

ALGORITMI SU GRAFI

```
def union(self, x, y):
    # Unisce gli insiemi che contengono x e y
    root_x = self.find(x)
    root_y = self.find(y)

    # Se hanno già lo stesso rappresentante, sono nello stesso insieme
    if root_x == root_y:
        return False

    # Union by rank: attacca l'albero più basso sotto quello più alto
    if self.rank[root_x] < self.rank[root_y]:
        self.parent[root_x] = root_y
    elif self.rank[root_x] > self.rank[root_y]:
        self.parent[root_y] = root_x
    else:
        self.parent[root_y] = root_x
        self.rank[root_x] += 1

    return True
```

ALGORITMI SU GRAFI

```
def kruskal(n, edges):
    """
    Calcola il Minimum Spanning Tree, cioè l'albero ricoprente minimo.

    Parametri:
    n     = numero di vertici, numerati da 0 a n - 1
    edges = lista di archi nel formato (peso, u, v)

    Ritorna:
    mst          = lista degli archi scelti nell'albero ricoprente minimo
    total_weight = peso totale dell'MST
    """

    # Ordina gli archi in ordine crescente di peso
    edges.sort()

    uf = UnionFind(n)
    mst = []
    total_weight = 0
```

ALGORITMI SU GRAFI

```
    # Scorre gli archi dal più leggero al più pesante
    for peso, u, v in edges:
        # Se u e v appartengono a componenti diverse,
        # l'arco può essere aggiunto senza creare cicli
        if uf.union(u, v):
            mst.append((u, v, peso))
            total_weight += peso

        # Un MST di n vertici contiene esattamente n - 1 archi
        if len(mst) == n - 1:
            break

    return mst, total_weight


# Esempio d'uso
n = 4

edges = [
    (10, 0, 1),
    (6, 0, 2),
    (5, 0, 3),
    (15, 1, 3),
    (4, 2, 3)
]
```

ALGORITMI SU GRAFI

```
mst, peso_totale = kruskal(n, edges)

print("Archi nell'MST:")
for u, v, peso in mst:
    print(f"{u} -- {v} con peso {peso}")

print("Peso totale:", peso_totale)
```

ALGORITMI SU GRAFI

RIPASSO: UNION-FIND

Supponiamo di avere una collezione S di n elementi distinti. Per fissare le idee, supponiamo che gli n elementi distinti siano i numeri interi da 1 a n. Inizialmente, questi elementi sono organizzati in n insiemi disgiunti: ogni insieme contiene esattamente un elemento. In maggiore dettaglio, avremo gli n insiemi disgiunti {1}, {2}, … {n}. Ogni insieme ha un nome: senza perdere di generalità, assumiamo che inizialmente il nome dell'insieme {i} sia i. 1 <= i <= n. Nel problema union-find desideriamo mantenere una collezione di insiemi disgiunti durante una sequenza di operazioni del seguente tipo:

- Union(A, B): Unisci gli insiemi A e B in un unico insieme. Il nome dell'insieme ottenuto è A. Questa operazione distrugge i vecchi insiemi A e B.
- Find(x): Dato un elemento, restituisci il nome dell'insieme che lo contiene.

ALGORITMI SU GRAFI

RIPASSO: UNION-FIND

Abbiamo assunto che n fosse noto a priori. Definiamo adesso il problema union-find nel caso più generale, in cui il numero totale di elementi non è noto a priori. Inizialmente non ci sono né insiemi né elementi, ed i nuovi elementi sono introdotti dall'operazione:

- makeSet(x): crea un nuovo insieme {x}, contenente un nuovo elemento x. Il nome dell'insieme {x} è x.

Diremo quindi che il problema union-find consiste nel mantenere una collezione di insiemi disgiunti durante una qualsiasi sequenza di operazioni union, find e makeSet a partire dall'insieme vuoto.

ALGORITMI SU GRAFI

Algoritmo di Prim

Questo algoritmo non guarda gli archi globalmente, ma parte da un singolo nodo arbitrario e fa "crescere" l'albero un passo alla volta. Ad ogni iterazione, guarda tutti gli archi che collegano i nodi già visitati a quelli non ancora visitati e sceglie quello con il peso minore.

A differenza dell'algoritmo di Kruskal, mantiene sempre un unico albero blu T. All'inizio tale albero consisterà in un unico vertice, che possiamo scegliere arbitrariamente. Sia s il vertice scelto. Per far convergere l'albero blu verso un minimo albero ricoprente, l'algoritmo di Prim applica per (n-1) volte il seguente passo:

Scegli un arco di costo minimo incidente sull'albero blu T e coloralo di blu. Dopo (n-1) passi l'albero blu contiene tutti i vertici. A tal punto l'algoritmo termina e restituisce l'albero blu calcolato.

ALGORITMI SU GRAFI

```python
import heapq


def prim(graph, start=0):
    """
    Calcola il Minimum Spanning Tree usando l'algoritmo di Prim.

    Parametri:
    graph = grafo rappresentato come lista di adiacenza.
            graph[u] contiene coppie (peso, v), dove:
            - u è il nodo corrente
            - v è un vicino di u
            - peso è il peso dell'arco u-v

    start = nodo da cui iniziare

    Ritorna:
    mst = lista degli archi scelti nell'albero ricoprente minimo
    total_weight = peso totale dell'MST
    """

    n = len(graph)
```

ALGORITMI SU GRAFI

```
# visited[u] è True se il nodo u è già stato inserito nell'MST
visited = [False] * n

# Coda di priorità degli archi candidati.
# Ogni elemento ha forma (peso, nodo_corrente, nodo_precedente)
heap = [(0, start, -1)]

mst = []
total_weight = 0

while heap and len(mst) < n - 1:
    peso, u, padre = heapq.heappop(heap)

    # Se il nodo è già stato visitato, ignoro questo arco
    if visited[u]:
        continue

    # Inserisco il nodo nell'MST
    visited[u] = True
    total_weight += peso

    # Se padre != -1, allora esiste un arco reale da aggiungere all'MST
    # Il primo nodo non ha padre, quindi non aggiunge archi
    if padre != -1:
        mst.append((padre, u, peso))
```

ALGORITMI SU GRAFI

```
        # Aggiungo alla coda tutti gli archi che portano a nodi non visitati
        for peso_vicino, v in graph[u]:
            if not visited[v]:
                heapq.heappush(heap, (peso_vicino, v, u))

    return mst, total_weight


# Esempio d'uso
graph = [
    [(10, 1), (6, 2), (5, 3)],    # vicini del nodo 0
    [(10, 0), (15, 3)],           # vicini del nodo 1
    [(6, 0), (4, 3)],             # vicini del nodo 2
    [(5, 0), (15, 1), (4, 2)]     # vicini del nodo 3
]

mst, peso_totale = prim(graph, start=0)

print("Archi nell'MST:")
for u, v, peso in mst:
    print(f"{u} -- {v} con peso {peso}")

print("Peso totale:", peso_totale)
```

ALGORITMI SU GRAFI

Flusso Massimo (Maximum Flow)

Cambiamo completamente scenario. Immagina una rete di tubature dell'acqua o una rete autostradale. Ogni arco ha una capacità massima (es. quanti litri d'acqua al secondo possono passare in quel tubo). Il problema del Flusso Massimo consiste nel calcolare quanta acqua (o traffico) al massimo può viaggiare da un nodo sorgente (S) a un nodo pozzo (T) senza superare i limiti di capacità dei canali.

Per risolverlo si usano algoritmi basati sul concetto di cammini aumentanti (percorsi da S a T in cui c'è ancora della capacità residua disponibile):

ALGORITMI SU GRAFI

Algoritmo di Ford-Fulkerson

Cerca un cammino aumentante qualsiasi da S a T, ci fa passare il massimo flusso possibile, aggiorna le capacità residue e ripete l'operazione finché non ci sono più cammini disponibili.
Il limite: Non specifica come scegliere questo cammino. Se si sceglie un cammino inefficiente, l'algoritmo può impiegare moltissimo tempo (la complessità dipende dal valore del flusso massimo stesso, il che è rischioso se le capacità sono numeri enormi).

Algoritmo di Edmonds-Karp

È una variante specifica e più intelligente di Ford-Fulkerson. Stabilisce una regola ferrea: per trovare il cammino aumentante si deve usare una visita in ampiezza (BFS).
Il vantaggio: Scegliendo sempre il cammino più corto in termini di numero di archi, Edmonds-Karp garantisce che l'algoritmo termini in un tempo strettamente legato

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

